



Faculté des Sciences et des Technologies

Laboratoire N° 4 _ Modélisation Mathématiques

Niveau : Licence III

Etudiant : Donsam Jean Gabard NOEL

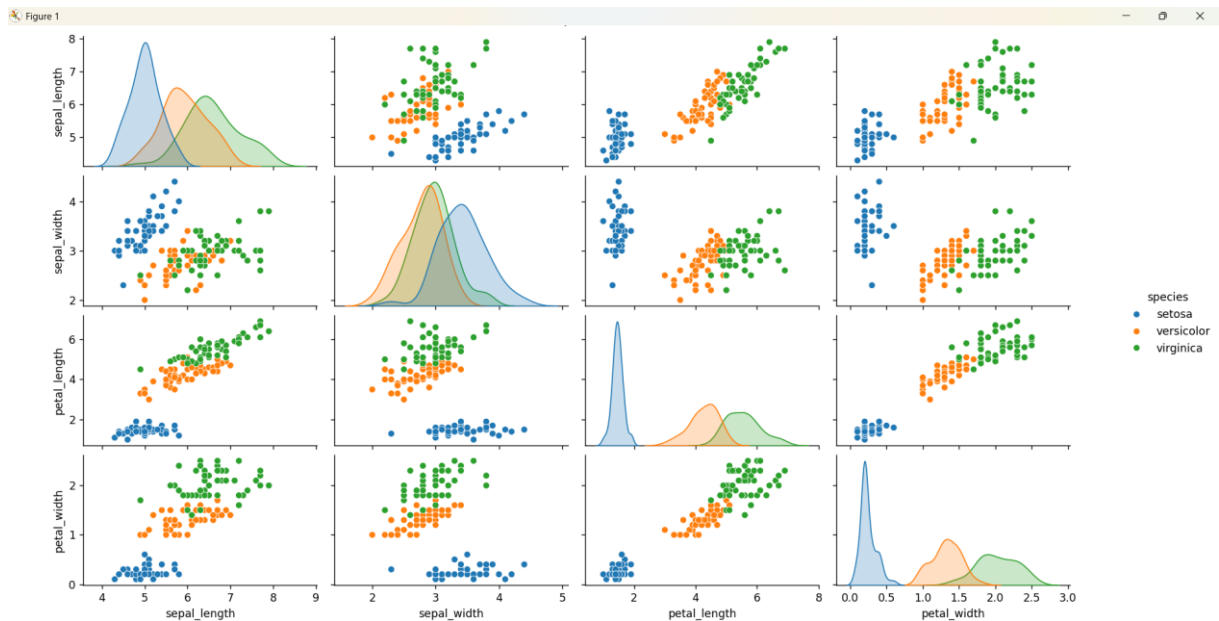
Professeur : Ismaël SAINT ARMOUR

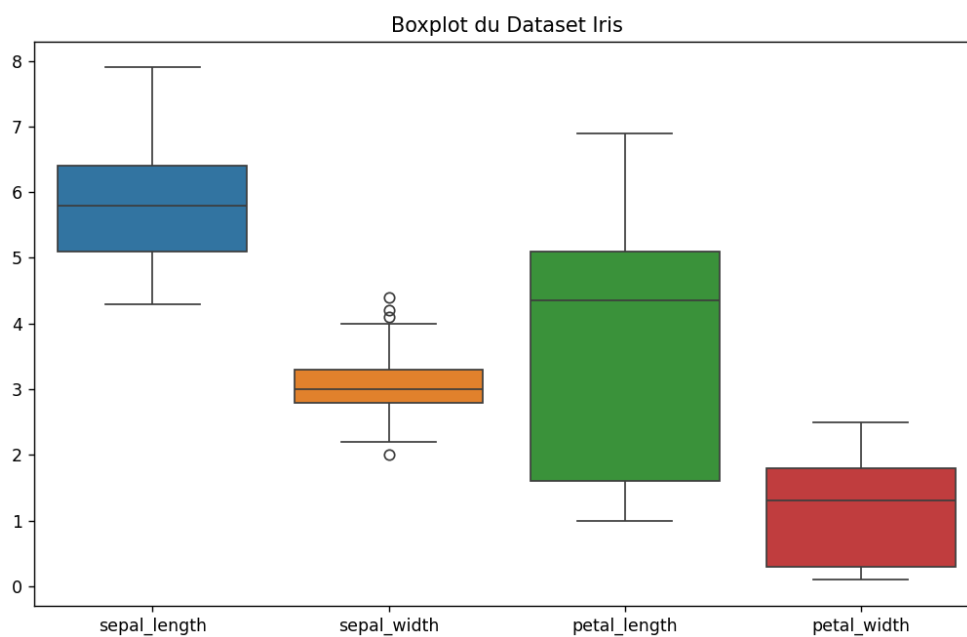
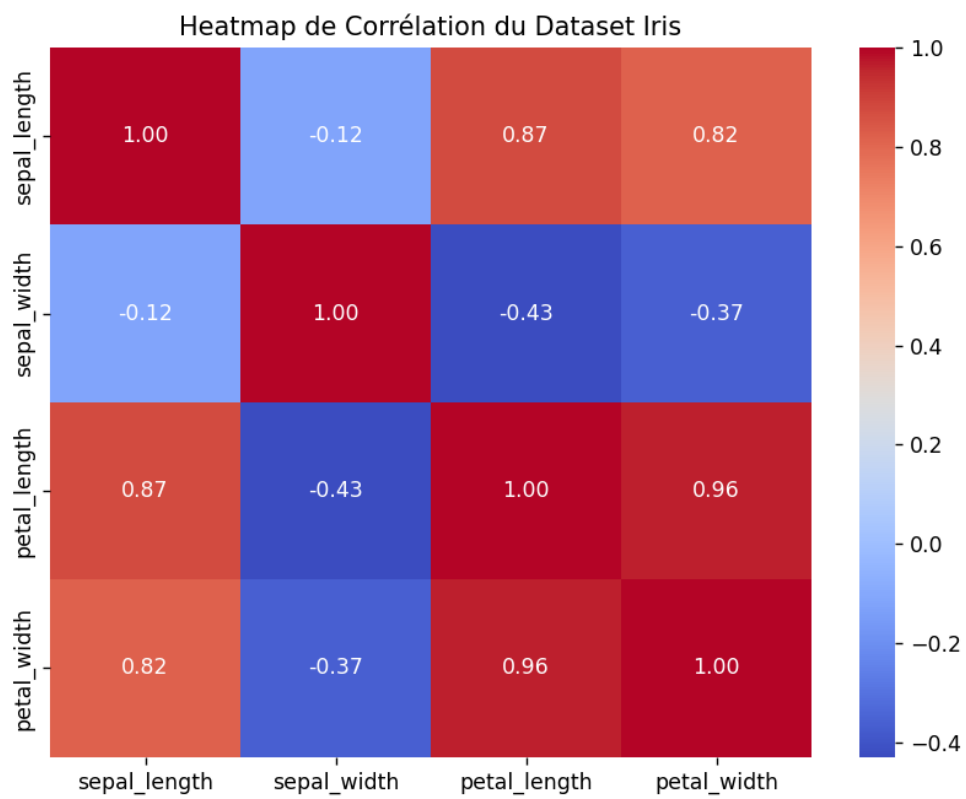
Date : 25 avril 2026

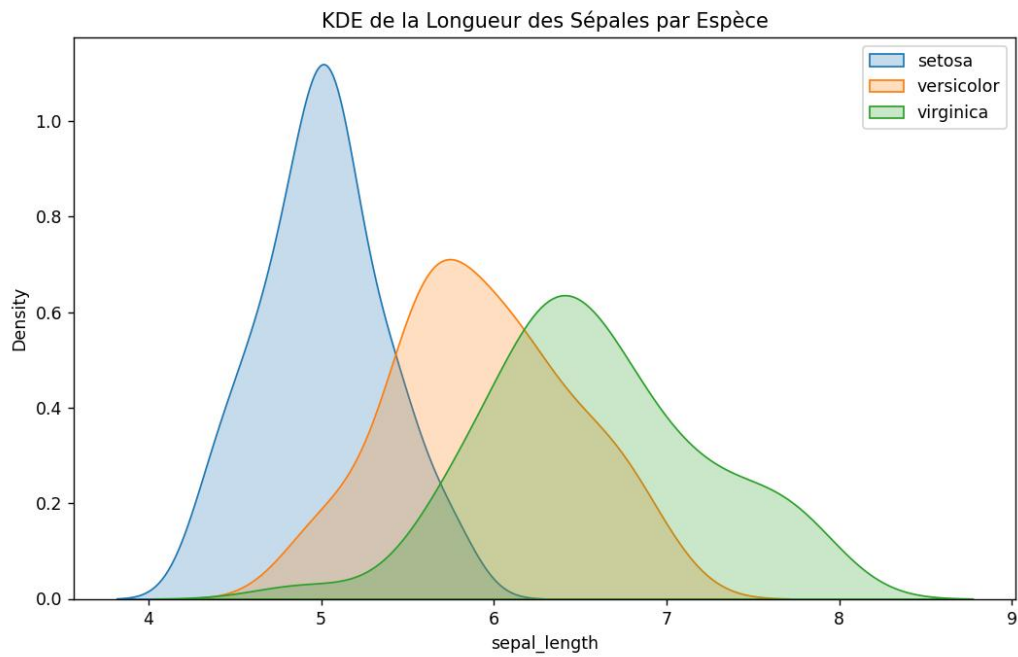
Travaux Dirigés

TD 1 - Analyse complète d'un dataset (EDA avancée)

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 # Charger le dataset Iris
5 df_iris = sns.load_dataset("iris")
6
7 # Pairplot
8 pairplot = sns.pairplot(df_iris, hue="species")
9 pairplot.fig.suptitle("Pairplot du Dataset Iris", y=1.02)
10 plt.show()
11
12 # Heatmap de corrélation
13 corr = df_iris.drop('species', axis=1).corr()
14 plt.figure(figsize=(8, 6))
15 sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
16 plt.title("Heatmap de corrélation du Dataset Iris")
17 plt.show()
18
19 # Boxplot
20 plt.figure(figsize=(10, 6))
21 sns.boxplot(data=df_iris.drop('species', axis=1))
22 plt.title("Boxplot du Dataset Iris")
23 plt.show()
24
25 # KDE
26 plt.figure(figsize=(10, 6))
27 for species in df_iris['species'].unique():
28     subset = df_iris[df_iris['species'] == species]
29     sns.kdeplot(data=subset['sepal_length'], label=species, fill=True)
30 plt.title("KDE de la Longueur des Sépales par Espèce")
31 plt.legend()
32 plt.show()
```





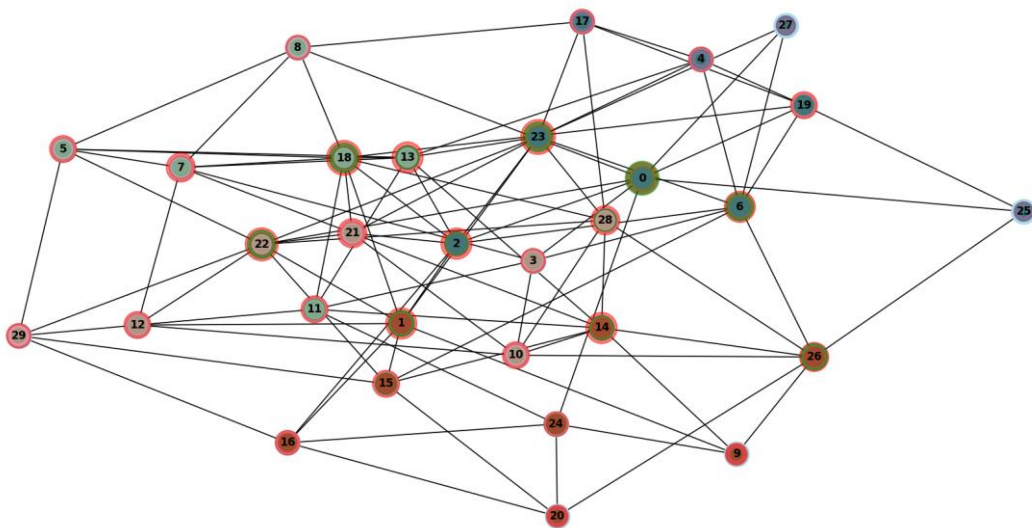


TD 2 — Visualisation de réseau social

```

1 import networkx as nx
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Créer un graphe de réseau social fictif avec 30 utilisateurs et des relations aléatoires
6 np.random.seed(42)
7 G = nx.erdos_renyi_graph(30, 0.2)
8
9 # Positionner les nœuds pour le dessin
10 pos = nx.spring_layout(G)
11
12 # Afficher le réseau
13 plt.figure(figsize=(10, 8))
14 nx.draw(G, pos, with_labels=True, node_color='lightblue', node_size=500, font_size=10, font_weight='bold')
15 plt.title("Réseau Social Fictif")
16
17 # Calculer et afficher
18 # le degré des nœuds
19 degrees = dict(G.degree())
20 nx.draw_networkx_nodes(G, pos, nodelist=degrees.keys(), node_size=[v * 100 for v in degrees.values()], node_color='red', alpha=0.5)
21
22 # La centralité d'intermédiation
23 betweenness = nx.betweenness_centrality(G)
24 nx.draw_networkx_nodes(G, pos, nodelist=betweenness.keys(), node_size=[v * 10000 for v in betweenness.values()], node_color='green', alpha=0.5)
25
26 # Les communautés
27 communities = nx.algorithms.community.greedy_modularity_communities(G)
28 community_colors = {node: i for i, comm in enumerate(communities) for node in comm}
29 nx.draw_networkx_nodes(G, pos, nodelist=community_colors.keys(), node_color=list(community_colors.values()), cmap=plt.cm.tab20, alpha=0.5)
30
31 # Afficher le graphe final
32 plt.show()

```



TD 3 — Simulation de propagation (virus ou info)

```

1 import networkx as nx
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Créer un graphe aléatoire pour la simulation
6 np.random.seed(42)
7 G_propagation = nx.erdos_renyi_graph(50, 0.1)
8
9 # Initialiser l'état des nœuds
10 state = {node: 0 for node in G_propagation.nodes}
11
12 # Infecter un nœud initial
13 initial_infected = np.random.choice(list(G_propagation.nodes))
14 state[initial_infected] = 1
15
16 # Fonction pour propager l'infection
17 def propagate(G, state, steps=5):
18     history = [dict(state)]
19     for _ in range(steps):
20         new_state = state.copy()
21         for node in G.nodes():
22             if state[node] == 1:
23                 for neighbor in G.neighbors(node):
24                     if state[neighbor] == 0 and np.random.rand() < 0.3:
25                         new_state[neighbor] = 1
26         history.append(new_state)
27         state = new_state
28     return history
29
30 # Simuler la propagation
31 history = propagate(G_propagation, state)
32
33 # Visualiser l'évolution
34 plt.figure(figsize=(15, 10))
35 for i, step in enumerate(history):
36     plt.subplot(2, 3, i+1)
37     pos = nx.spring_layout(G_propagation)
38     nx.draw(G_propagation, pos, nodelist=step.keys(), node_color=["red" if step[node] == 1 else "green" for node in G_propagation.nodes], with_labels=True)
39     plt.title(f"Étape {i}")
40
41 plt.suptitle("Simulation de Propagation d'un Virus dans un Réseau", y=1.02)
42 plt.tight_layout()
43 plt.show()
44
45 # Courbe des infectés vs temps
46 times = list(range(len(history)))
47 infected_counts = [sum(s == 1 for s in step.values()) for step in history]
48
49 plt.figure(figsize=(8, 6))
50 plt.plot(times, infected_counts, marker='o')
51 plt.title("Nombre d'Infectés vs Temps")
52 plt.xlabel("Temps")
53 plt.ylabel("Nombre d'Infectés")
54 plt.xticks(times)
55 plt.grid(True)
56 plt.show()

```

